

Отчет по Лабораторной работе 2
по курсу программирование в системах баз данных

Студентов группы КЕ-23-02
Баранов Дмитрий Александрович
Голошумов Михаил Дмитриевич

Исходное задание
"БиоСборка" (Генетический конструктор)

1) Цель

Игра-симулятор, где нужно скрещивать виртуальных существ, чтобы получить организм с заданными свойствами.

2) Уровни требований

На удовлетворительно (3)

1. **Существа:** 5 базовых видов с 3-4 признаками (цвет, размер, есть ли крылья, тип питания).
2. **Генетика (упрощённая):** Каждый признак — один "ген" с двумя аллелями (доминантный А, рецессивный а).
3. **Скрещивание:** Выбор двух существ. Система вычисляет возможное потомство по менделевским правилам.
4. **Задания:** "Получи рыбку зелёного цвета с носом как у рыбы-пилы". Нужно подобрать родителей.

На хорошо (4)

1. **Сложная генетика:** Неполное доминирование, сцепленные гены, мутации (случайное изменение гена).
2. **Эволюционные линии:** Цель — от волко-собаки вывести идеальную породу по заказу "клиента".
3. **Эксперименты:** Возможность влиять на мутации (облучение, химикаты) с риском.

Перечень основных объектов, необходимых для создания игры "БиоСборка".

Данный перечень носит предварительный характер и может быть изменен в ходе реализации. После выполнения 3-ей лабораторной работы данный отчет может быть доработан, при необходимости.

Для реализации игры нам будет необходимо создать служебные таблицы:

Подробнее содержимое таблиц описано в таблице 1.

Таблица 1 – описание таблиц схемы базы данных

Таблица	Столбец	Тип данных	Ограничения целостности	Описание
Пользователи	Идентификатор пользователя	NUMBER	PK	Уникальный идентификатор пользователя
	Имя пользователя	VARCHAR(255)	NOT NULL	Имя пользователя
	Логин	VARCHAR(20)	NOT NULL, UNIQUE, только буквы строчные латинского алфавита, символы "1234567890_" не могут быть первыми, но могут быть использованы	Логин для входа в систему
	Пароль	CHAR(512)	NOT NULL	Хеш пароля
Сессии	Идентификатор сессии	NUMBER	PK	Уникальный идентификатор сессии
	Идентификатор пользователя	NUMBER	NOT NULL	Ссылка на уникальный идентификатор пользователя
	Статус	CHAR(1)	NOT NULL	Активна, завершена
	Дата начала сессии	DATE	NOT NULL	Заполняется при инициализации сессии
	Дата конца сессии	DATE		Заполняется при завершении сессии
Лаборатория	Идентификатор лаборатории	NUMBER	PK	Уникальный идентификатор лаборатории
	Идентификатор пользователя	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор пользователя
	Идентификатор сессии	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор сессии
	Кошелек	NUMBER		Число монет

	Рейтинг лаборатории	NUMBER		Зависит от выполнения заданий и проведения экспериментов
	Количество существ в лаборатории	NUMBER		Рассчитывается из количества текущих существ в лаборатории
	Количество активных заданий	NUMBER		Рассчитывается из количества текущих существ в задании
	Количество выполненных заданий	NUMBER		Число выполненных заданий
	Количество проведенных экспериментов	NUMBER		Число проведенных экспериментов
Ген	Идентификатор гена	NUMBER	PK	Уникальный идентификатор гена
	Тип гена	varchar2(50)	NOT NULL	Тип гена отвечает за признак, которой он кодирует
	Тип существа	NUMBER	NOT NULL, DEFAULT 0, от 1 до 6	0 — универсальный ген (для всех существ); 1 – 6 — привязка к конкретному типу существ (хрящевые рыбы, костные рыбы и т.д.)
	Тип доминирования	varchar2(20)	NOR NULL, DEFAULT 'FULL', также может быть 'INCOMPLETE' или 'CODOMINANT'	Необходим для выбора типа доминирования.
	Название гена	VARCHAR2(50)	NOT NULL	Параметр, отвечающий за название гена
	Описание гена	varchar2(255)		Приводится текст, за что отвечает ген
Аллель	Идентификатор аллеля	NUMBER	PK	Уникальный идентификатор
	Идентификатор гена	NUMBER	NOT NULL	Ссылка на идентификатор гена
	Доминантность	NUMBER		Доминантный или рецессивный
	Описание аллеля	varchar2(255)		Определенное значение признака

	Значение признака	NUMBER	NOT NULL	Число, используемое для расчета значения признака
Эксперимент	Идентификатор лаборатории	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор лаборатории
	Идентификатор эксперимента	NUMBER	PK	Уникальный идентификатор эксперимента
	Родитель 1	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор существа, которое выступает в качестве первого родительского существа (если эксперимент с мутацией, то может быть единственным)
	Родитель 2	NUMBER	FK	Ссылка на уникальный идентификатор существа, которое выступает в качестве второго родительского существа
	Идентификатор мутации	NUMBER	FK	Ссылка на уникальный идентификатор мутации, может не использоваться в эксперименте
	Идентификатор потомка	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор существа, которое получилось в результате эксперимента
	Тип эксперимента	varchar2(255)		Скращивание, применение мутации на существо, скращивание с мутагеном
Существо	Идентификатор существа	NUMBER	PK	Уникальный идентификатор существа

	Идентификатор лаборатории	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор лаборатории
	Имя существа	VARCHAR2(255)	NOT NULL	Параметр, отвечающий за имя существа
	Описание признаков	NUMBER		Значения признаков, формирующие фенотип существа
Мутация	Идентификатор мутации	NUMBER	PK	Уникальный идентификатор мутации
	Имя мутации	VARCHAR2(50)	NOT NULL	Параметр, отвечающий за имя мутации
	Тип мутации	NUMBER		Числовой идентификатор мутации
	Описание мутации	VARCHAR2(255)		Что произойдет при применении данной мутации
	Стоимость мутации	NUMBER		За сколько монет ее можно купить в магазине
	Рейтинг мутации	NUMBER		Влияние на рейтинг при проведении эксперимента
Задание	Идентификатор задания	NUMBER	PK	Уникальный идентификатор задания
	Название задания	VARCHAR2(100)	NOT NULL	Параметр, отвечающий за название задания
	Описание задания	VARCHAR2(255)		Что нужно сделать
	Рейтинг задания	NUMBER		На сколько изменится рейтинг при выполнении задания
	Стоимость задания	NUMBER		Сколько монет даст выполнение задания
Задание в лаборатории	Идентификатор лаборатории	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор лаборатории
	Идентификатор задания	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор задания

	Статус задания	varchar2(25)		Выполнено, в процессе выполнения
Мутация в лаборатории	Идентификатор лаборатории	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор лаборатории
	Идентификатор мутации	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор мутации
	Количество мутаций	NUMBER		Количество доступных мутаций
Генотип	Идентификатор генотипа	NUMBER	PK	Уникальный идентификатор генотипа
	Идентификатор существа	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор существа
	Идентификатор гена	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор гена
	Идентификатор первого аллеля	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор аллеля
	Идентификатор второго аллеля	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор аллеля
Маркер задания	Идентификатор задания	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор задания, которое выполняется
	Идентификатор аллеля	NUMBER	FK, NOT NULL	Ссылка на уникальный идентификатор аллеля, значение которого является целевым

Ниже приведена предварительная схема базы данных, необходимая для реализации игры (рисунок 1).

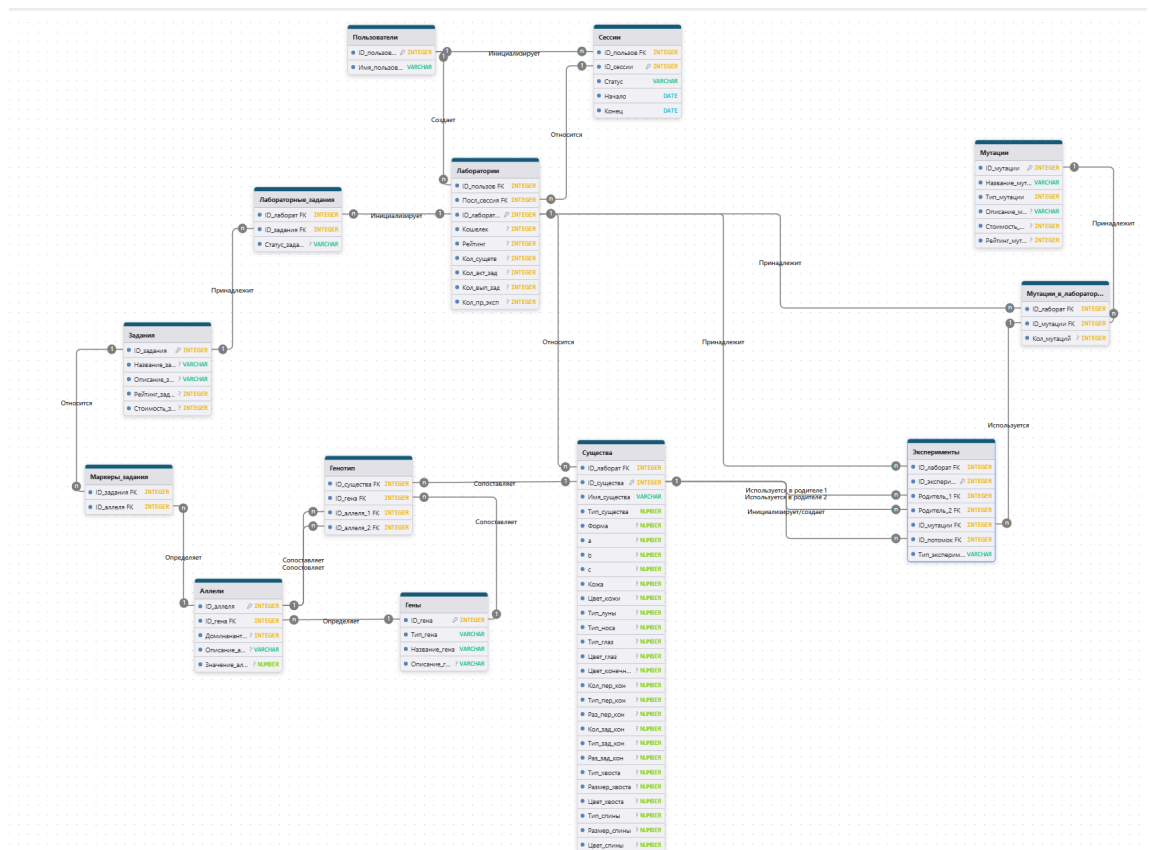


Рисунок 1 - Полноатрибутная модель базы данных

Все программные модули объединены в пакет PKG_GENETICS_GAME.
Спецификация пакета:

```
CREATE OR REPLACE PACKAGE PKG_GENETICS_GAME AS
    PROCEDURE register_user(
        p_username VARCHAR2,
        p_login VARCHAR2,
        p_password VARCHAR2,
        p_user_id OUT INTEGER
    );
    FUNCTION login_user(
        p_login VARCHAR2,
        p_password VARCHAR2
    ) RETURN VARCHAR2;
    PROCEDURE logout_user(p_session_token VARCHAR2);
    PROCEDURE update_user_profile(
        p_user_id INTEGER,
        p_username VARCHAR2 DEFAULT NULL,
        p_password VARCHAR2 DEFAULT NULL
    );
    FUNCTION hash_password(p_password VARCHAR2) RETURN VARCHAR2;
    PROCEDURE load_lab(
        p_session_token VARCHAR2,
        p_lab_id INTEGER
    );
    PROCEDURE start_new_lab(p_lab_id OUT INTEGER);
    FUNCTION list_user_labs(p_user_id INTEGER) RETURN SYS_REFCURSOR;
    PROCEDURE exit_lab(p_lab_id INTEGER);
```

```

PROCEDURE switch_lab(
    p_session_token VARCHAR2,
    p_new_lab_id     INTEGER
);

PROCEDURE show_lab_stats(p_lab_id INTEGER);
PROCEDURE get_lab_stats(
    p_lab_id          INTEGER,
    p_wallet           OUT INTEGER,
    p_rating           OUT INTEGER,
    p_creature_count   OUT INTEGER,
    p_active_tasks     OUT INTEGER,
    p_completed_tasks  OUT INTEGER,
    p_experiments      OUT INTEGER
);
PROCEDURE show_creatures(p_lab_id INTEGER);
FUNCTION get_creatures_cursor(p_lab_id INTEGER) RETURN SYS_REFCURSOR;
FUNCTION get_genotype_cursor(p_creature_id INTEGER) RETURN SYS_REFCURSOR;
FUNCTION get_phenotype(p_creature_id INTEGER) RETURN VARCHAR2;
FUNCTION get_dominant_allele(
    p_creature_id INTEGER,
    p_gene_id      INTEGER
) RETURN VARCHAR2;
FUNCTION get_inherited_allele(
    p_parent_id INTEGER,
    p_gene_id   INTEGER
) RETURN INTEGER;
FUNCTION get_linked_allele_set(
    p_creature_id   INTEGER,
    p_linkage_group INTEGER
) RETURN VARCHAR2;
FUNCTION calculate_punnett_probabilities(
    p_parent1_id INTEGER,
    p_parent2_id INTEGER,
    p_gene_id    INTEGER
) RETURN SYS_REFCURSOR;
PROCEDURE crossbreed(
    p_lab_id          INTEGER,
    p_parent1_id      INTEGER,
    p_parent2_id      INTEGER,
    p_offspring_name  VARCHAR2,
    p_offspring_id    OUT INTEGER
);

PROCEDURE rename_creature(
    p_creature_id   INTEGER,
    p_new_name      VARCHAR2
);

PROCEDURE apply_mutation(
    p_creature_id INTEGER,
    p_mutation_id INTEGER
);
FUNCTION buy_mutation(
    p_lab_id          INTEGER,
    p_mutation_id     INTEGER
) RETURN BOOLEAN;
PROCEDURE make_experiment(
    p_lab_id          INTEGER,
    p_parent1_id      INTEGER,
    p_parent2_id      INTEGER,
    p_mutation_id     INTEGER DEFAULT NULL,
    p_offspring_name  VARCHAR2,

```



```

        p_offspring_id    OUT INTEGER
    );
    PROCEDURE show_tasks(p_lab_id INTEGER);
    FUNCTION check_task(
        p_lab_id          INTEGER,
        p_task_id         INTEGER,
        p_creature_id     INTEGER
    ) RETURN INTEGER;
    PROCEDURE complete_task(
        p_lab_id          INTEGER,
        p_task_id         INTEGER,
        p_creature_id     INTEGER
    );
    PROCEDURE apply_mutagen(
        p_creature_id     INTEGER,
        p_mutagen_type    INTEGER,
        p_new_creature_id OUT INTEGER
    );
    PROCEDURE show_mutation_history(p_lab_id INTEGER);
    FUNCTION show_mutation_shop RETURN SYS_REFCURSOR;
    PROCEDURE generate_starting_creatures(p_lab_id INTEGER);
    PROCEDURE create_creature_of_type(
        p_lab_id          INTEGER,
        p_species_type    INTEGER,
        p_variant         INTEGER,
        p_creature_id     OUT INTEGER
    );
    PROCEDURE delete_lab(p_lab_id INTEGER);

END PKG_GENETICS_GAME;
/

```

Далее подробнее рассмотрим описание функционала программных модулей.

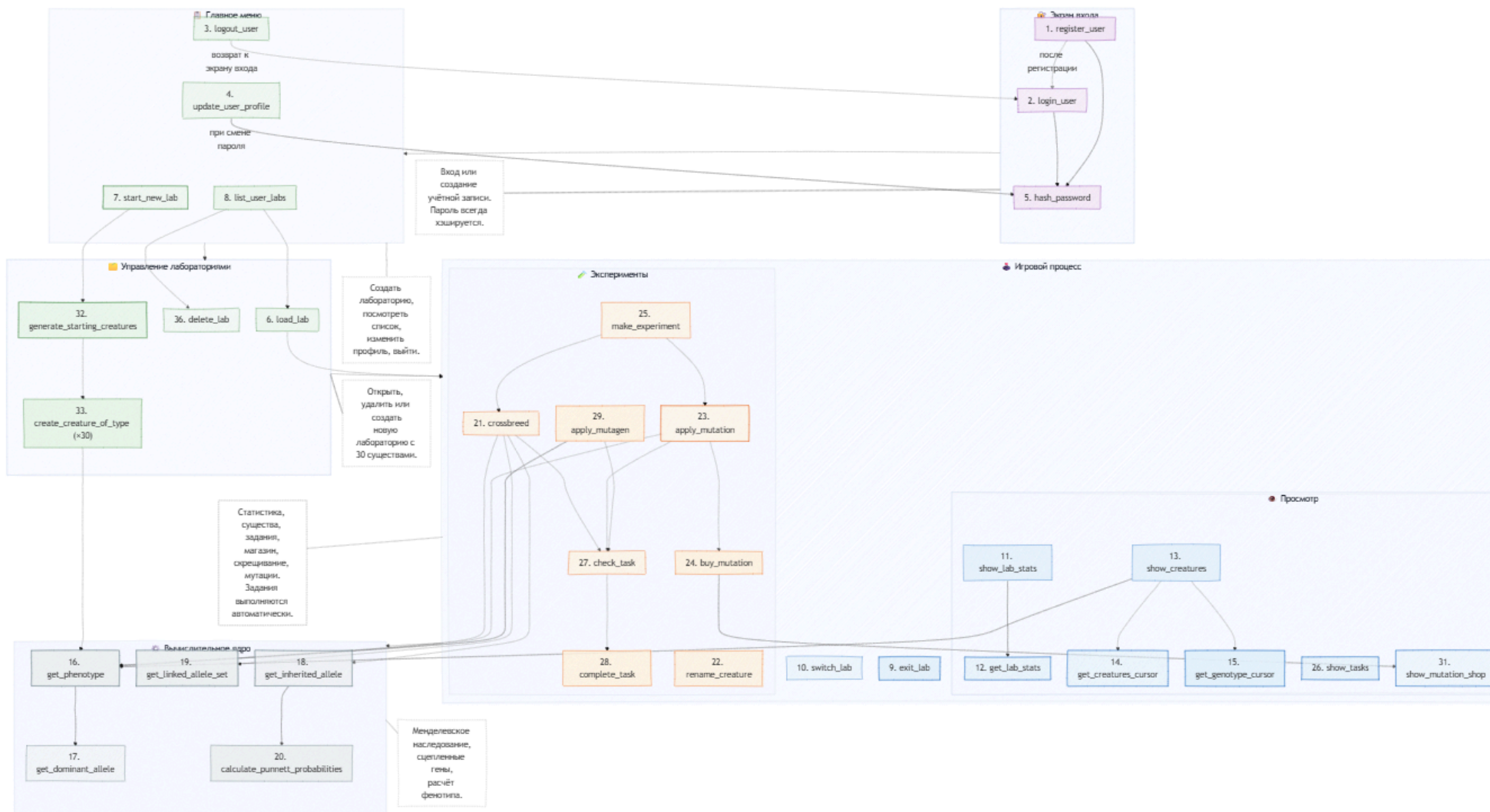


Рисунок 2 - Схема программной реализации игры «Генетический конструктор» на Oracle PL/SQL

Приведем подробное описание процедур:

1. Процедура `register_user(p_username, p_login, p_password, p_user_id OUT)`

Общая суть: регистрация нового игрока.

С точки зрения внутренней логики: проверяется уникальность переданного `p_login` (строчные латинские буквы, цифры, символ подчёркивания; первый символ – обязательно буква). Пароль хэшируется вызовом `hash_password`, после чего в таблицу «Пользователи» добавляется запись с именем, логином и хэшем. Идентификатор созданной записи возвращается в `p_user_id`. Транзакция фиксируется.

С точки зрения игрока: после успешной регистрации можно сразу входить в систему.

2. Функция `login_user(p_login, p_password) RETURN VARCHAR2`

Общая суть: аутентификация пользователя и создание сессии приложения.

С точки зрения внутренней логики: по логину находится запись в таблице «Пользователи», вычисляется хэш от переданного пароля. Если хэши совпадают, в таблице «Сессии» создаётся новая запись со статусом `ACTIVE`, текущим временем начала и идентификатором пользователя. Возвращается уникальный токен сессии (например, идентификатор сессии). Глобальные переменные пакета инициализируются текущим пользователем и сессией. При неверных учётных данных возвращается `NULL`.

С точки зрения игрока: после ввода логина и пароля система либо пускает в игру, возвращая токен, либо сообщает об ошибке.

3. Процедура `logout_user(p_session_token VARCHAR2)`

Общая суть: завершение сессии приложения.

С точки зрения внутренней логики: по токену находится активная сессия, в таблице «Сессии» проставляются дата окончания (SYSDATE) и статус CLOSED. Глобальные переменные пакета очищаются.

С точки зрения игрока: безопасный выход из учётной записи, все активные операции прекращаются.

4. Процедура update_user_profile(p_user_id, p_username, p_password)

Общая суть: редактирование данных профиля.

С точки зрения внутренней логики: если передан p_username, обновляется имя пользователя. Если передан p_password, он хэшируется и сохраняется в таблице «Пользователи» вместо старого хэша. Неизменённые параметры можно оставить NULL, тогда соответствующие поля не обновляются.

Для игрока: позволяет сменить отображаемое имя или пароль без перерегистрации.

5. Функция hash_password(p_password) RETURN VARCHAR2

Общая суть: вычисление SHA-256 хэша от пароля.

С точки зрения внутренней логики: используется стандартный пакет DBMS_CRYPTO, результат возвращается в виде 512-битной строки (64 символа в шестнадцатеричной кодировке).

С точки зрения игрока: напрямую не вызывается, обеспечивает безопасное хранение паролей.

6. Процедура load_lab(p_session_token VARCHAR2, p_lab_id INTEGER)

Общая суть работы: установка указанной лаборатории как активной для текущего пользователя после проверки, что она принадлежит ему.

С точки зрения внутренней логики: по переданному p_session_token из таблицы «Сессии» определяется идентификатор пользователя (v_user_id).

Проверяется существование лаборатории с ID_лаборатории = p_lab_id и ID_пользователя = v_user_id.

Если запись не найдена, генерируется исключение с сообщением «Лаборатория не принадлежит текущему пользователю».

Если лаборатория существует, в глобальной переменной пакета фиксируется `g_active_lab_id := p_lab_id` (активная лаборатория для текущей сессии пользователя).

При необходимости также может обновляться запись в таблице «Лаборатория» — ссылка на текущую сессию (`ID_сессии_FK`), чтобы зафиксировать, что пользователь работает именно с этой лабораторией.

С точки зрения игрока: пользователь выбирает одну из своих лабораторий (из списка, полученного через `list_user_labs`) и «загружает» её. После успешной загрузки все дальнейшие действия (скрещивание, просмотр существ, задания) выполняются в контексте выбранной лаборатории.

7. Процедура `start_new_lab(p_lab_id OUT INTEGER)`;

Общая суть работы: создание новой игровой лаборатории для текущего пользователя, идентифицированного по активной сессии.

С точки зрения внутренней логики: идентификатор пользователя извлекается из глобального контекста пакета (установлен при логине). Создаётся запись в таблице «Лаборатории», связывающая её с пользователем и текущей сессией (`ID_сессии_FK`). Начальные значения: кошелек = 1000, рейтинг = 0, все счётчики обнулены. Вызывается `generate_starting_creatures` для получения 30 стартовых существ. Генерируются 3 стартовых задания, и обновляются счётчики лаборатории. Транзакция фиксируется.

С точки зрения игрока: после вызова процедуры пользователь получает новую лабораторию с уникальным идентификатором (`p_lab_id`), в которой уже есть 30 существ разных биологических типов и несколько доступных заданий. Игрок может сразу приступить к экспериментам и выполнению заказов.

8. Функция `list_user_labs(p_user_id INTEGER) RETURN
SYS_REFCURSOR`

Общая суть работы: возвращает курсор со списком всех лабораторий, принадлежащих пользователю.

С точки зрения внутренней логики: выполняется запрос:

```
SELECT ID_лаборатории, ID_сессии_FK, Кошелёк,  
Рейтинг, Кол_сущетв, Кол_акт_зад, Кол_вып_зад,  
Кол_пр_эксп
```

```
FROM Лаборатории
```

```
WHERE ID_пользователя = p_user_id;
```

Открытый курсор возвращается вызывающему коду.

С точки зрения игрока: При входе в систему или в разделе «Мои лаборатории» игрок видит перечень своих лабораторий с основной статистикой. Он может выбрать одну для загрузки или переключения, а также создать новую.

9. Процедура exit_lab(p_lab_id)

Общая суть работы: завершение текущей игровой сессии лаборатории.

С точки зрения внутренней логики: находит последнюю активную сессию, связанную с лабораторией (через поле «Посл_сессия_FK»), и обновляет в таблице «Сессии» колонки «Конец» = SYSDATE и «Статус» = 'CLOSED'. Сама лаборатория остаётся в базе данных, но дальнейшие эксперименты будут возможны только после новой загрузки и создания свежей сессии.

С точки зрения игрока: при завершении работы с лабораторией время окончания фиксируется. В следующий раз при входе можно будет выбрать сохранённую лабораторию из списка или начать новую.

10. Процедура `switch_lab(p_session_token VARCHAR2, p_new_lab_id INTEGER)`

Общая суть работы: переключение между лабораториями в рамках одной пользовательской сессии без повторного входа.

С точки зрения внутренней логики: выполняется тот же алгоритм, что и в `load_lab`: идентификация пользователя по токenu, проверка принадлежности лаборатории.

При успехе происходит смена активной лаборатории: `g_active_lab_id := p_new_lab_id`.

Предыдущая лаборатория остаётся в базе неизменной (все её данные сохраняются). Одновременно активной может быть только одна лаборатория.

Если пользователь хочет работать с несколькими лабораториями параллельно, он должен последовательно вызывать `switch_lab`, но в любой момент времени глобальный контекст указывает ровно на одну.

С точки зрения игрока: удобное переключение между сохранёнными лабораториями без необходимости выходить и заходить заново. В интерфейсе это может быть реализовано выпадающим списком или панелью вкладок.

Ограничения: в рамках одной сессии активна только одна лаборатория.

При переключении все незавершённые транзакции предыдущей лаборатории должны быть зафиксированы.

11. Процедура `show_lab_stats(p_lab_id)`

Общая суть работы: вывод в консоль актуальной статистики лаборатории.

С точки зрения внутренней логики: процедура объявляет локальные переменные для кошелька, рейтинга и счётчиков, вызывает

get_lab_stats(p_lab_id, wallet, rating, creature_count, active_tasks, completed_tasks, experiments), после чего выводит полученные значения через DBMS_OUTPUT.PUT_LINE. Сама статистика не пересчитывается — всю работу выполняет get_lab_stats.

С точки зрения игрока: при вызове в интерфейсе игрок видит точные показатели лаборатории на текущий момент (число существ, заданий, монет, рейтинг и т.д.), обновлённые по фактическому содержимому коллекции.

12. Процедура get_lab_stats(p_lab_id IN INTEGER, p_wallet OUT INTEGER, p_rating OUT INTEGER, p_creature_count OUT INTEGER, p_active_tasks OUT INTEGER, p_completed_tasks OUT INTEGER, p_experiments OUT INTEGER)

Общая суть работы: перерасчёт показателей лаборатории на основе данных из связанных таблиц, сохранение вычисленных значений в записи лаборатории и возврат их через выходные параметры.

С точки зрения внутренней логики: получает текущие значения кошелька и рейтинга прямым чтением из таблицы «Лаборатории»:

```
SELECT Кошелек, Рейтинг INTO p_wallet, p_rating
FROM Лаборатории WHERE ID_лаборатории = p_lab_id;
```

Эти поля не пересчитываются, так как изменяются только в ходе игровых действий (награды, покупки) и хранят актуальное состояние.

Вычисляет фактические значения счётчиков, выполняя агрегирующие запросы к дочерним таблицам:

Количество существ:

```
SELECT COUNT(*) INTO p_creature_count FROM
Существа WHERE ID_лаборат_FK = p_lab_id;
```

Количество активных заданий:

```
SELECT COUNT(*) INTO p_active_tasks FROM
Лабораторные_задания WHERE ID_лаборат_FK = p_lab_id
AND Статус_задания = 'ACTIVE';
```


Количество выполненных заданий:

```
SELECT COUNT(*) INTO p_completed_tasks FROM
Лабораторные_задания WHERE ID_лаборат_FK = p_lab_id
AND Статус_задания = 'COMPLETED';
```

Количество проведённых экспериментов:

```
SELECT COUNT(*) INTO p_experiments FROM
Эксперименты WHERE ID_лаборат_FK = p_lab_id;
```

Обновляет агрегатные столбцы в таблице «Лаборатории» только что вычисленными значениями, чтобы хранящая запись оставалась синхронной с реальностью:

```
UPDATE Лаборатории SET Кол_существ =
p_creature_count, Кол_акт_зад = p_active_tasks,
Кол_вып_зад = p_completed_tasks, Кол_пр_эксп =
p_experiments WHERE ID_лаборатории = p_lab_id;
```

Возвращает все показатели (включая кошелек и рейтинг) через OUT-параметры.

С точки зрения игрока: пользователь в любой момент может запросить актуальную статистику лаборатории, вызывая эту процедуру. Гарантируется, что цифры на экране всегда соответствуют реальному количеству существ, заданий и экспериментов, даже если в результате предыдущих действий агрегатные поля не были немедленно обновлены. Для игрока это выглядит как надёжное и согласованное отображение состояния лаборатории.

13. Процедура show_creatures(p_lab_id)

Общая суть работы: вывод списка всех существ лаборатории с их фенотипом.

С точки зрения внутренней логики: для каждого существа из таблицы «Существа» с фильтром по ID_лаборат_FK вызывается функция get_phenotype, результат выводится в консоль.

С точки зрения игрока: при текстовом интерфейсе можно увидеть перечень своих существ и их внешние признаки (фенотип). В GUI эта процедура не используется, вместо неё применяется курсор.

14. Функция `get_creatures_cursor(p_lab_id)` RETURN SYS_REFCURSOR

Общая суть работы: возвращает курсор с основными полями существ лаборатории для использования в графическом интерфейсе.

С точки зрения внутренней логики: открывается курсор с запросом:

```
SELECT ID_существа, Имя_существа, Тип_существа,  
get_phenotype(ID_существа) AS Фенотип FROM Существа  
WHERE ID_лаборат_FK = p_lab_id.
```

Курсор возвращается вызывающему коду.

С точки зрения игрока: позволяет интерфейсу отобразить таблицу или карточки существ, отображая имя, тип и фенотип.

15. Функция `get_genotype_cursor(p_creature_id)` RETURN SYS_REFCURSOR

Общая суть работы: возвращает курсор с полным генотипом существа.

С точки зрения внутренней логики: курсор объединяет таблицы «Генотип», «Гены», «Аллели» (дважды) и возвращает название гена, значения обоих аллелей и их доминантность.

С точки зрения игрока: при выборе существа в интерфейсе можно посмотреть, какие аллели у него по каждому гену, что помогает планировать скрещивания.

16. Функция `get_phenotype(p_creature_id)` RETURN VARCHAR2

Общая суть работы: вычисляет фенотип существа — конкретные значения всех его признаков (цвет, размер и т.д.) — на основе генотипа, записывает эти значения в соответствующие столбцы таблицы «Существа» и возвращает итоговое текстовое описание для удобства отображения.

С точки зрения внутренней логики: из таблицы «Генотип» для существа `p_creature_id` выбираются все записи (ген, первый аллель, второй аллель).

Для каждой пары аллелей загружаются из таблицы «Аллель»:

описание_1, описание_2 — текстовое описание признака;

доминантность_1, доминантность_2 — сила аллеля;

значение_1, значение_2 — числовое значение признака.

Из таблицы «Гены» для текущего гена извлекается тип доминирования ('FULL', 'INCOMPLETE', 'CODOMINANT').

Для каждого гена определяется итоговый проявляемый признак:

Если доминантность_1 > доминантность_2 → признак = описание_1.

Если доминантность_2 > доминантность_1 → признак = описание_2.

Если доминантности равны (гетерозигота):

При 'INCOMPLETE': вычисляется среднее (значение_1 + значение_2) /

2. В таблице «Аллель» ищется запись для того же гена, у которой Значение признака равно этому среднему. Если найдена — используется её Описание аллеля. Иначе — 'промежуточный(значение_признака_1 '/' значение_признака_2)'.
При 'CODOMINANT': признак = описание_1 || '/' || описание_2.

При 'FULL': признак = описание_1 (при равной доминантности оба аллеля дают одинаковый признак).

Вычисленное значение признака записывается в соответствующий столбец таблицы «Существа»:

```
UPDATE Существа
SET Цвет = CASE WHEN ген = 'Цвет' THEN признак
ELSE Цвет END,
    Размер = CASE WHEN ген = 'Размер' THEN признак
ELSE Размер
    ...
WHERE ID_существа = p_creature_id;
```

После обновления всех столбцов формируется итоговая строка-описание фенотипа и возвращается функцией.

С точки зрения игрока: при создании или изменении существа его фенотип полностью рассчитывается и сохраняется. В карточке существа в интерфейсе отображаются конкретные признаки (цвет, размер и т.д.), а также доступна сводная строка описания.

17. Функция `get_dominant_allele(p_creature_id, p_gene_id)` RETURN VARCHAR2

Общая суть работы: возвращает ID доминирующего аллеля для заданного гена существа (или оба через запятую при равной доминантности).

С точки зрения внутренней логики: находит в «Генотипе» два аллеля для указанного существа и гена, сравнивает их доминантность и возвращает соответствующий ID.

С точки зрения игрока: используется внутри других процедур (например, при построении фенотипа), напрямую игроку не требуется.

18. Функция `get_inherited_allele(p_parent_id, p_gene_id)` RETURN INTEGER

Общая суть работы: случайный выбор одного из двух аллелей родителя для передачи потомку (вероятность 50/50). Обеспечивает независимое менделевское наследование для генов, не входящих в группы сцепления.

С точки зрения внутренней логики: из таблицы «Генотип» для заданного родителя и гена извлекаются `ID\_аллеля\_1\_FK` и `ID\_аллеля\_2\_FK`. С помощью `DBMS\_RANDOM.VALUE` генерируется число от 0 до 1. Если оно меньше 0.5, возвращается `ID\_аллеля\_1\_FK`, иначе — `ID\_аллеля\_2\_FK`.

С точки зрения игрока: напрямую не вызывается, но результат её работы виден в генотипе потомка — признаки распределяются случайно, как в реальной генетике.

19. Функция `get_linked_allele_set(p_creature_id, p_linkage_group)` RETURN VARCHAR2

Общая суть работы: для группы сцепления возвращает строку с идентификаторами аллелей одного из двух гомологичных наборов хромосом родителя, которые должны передаваться потомку совместно, без разрыва. Моделирует наследование целой хромосомы.

С точки зрения внутренней логики: определяется список генов, принадлежащих данной группе сцепления (используется столбец `'linkage_group'` в таблице «Гены», если группа указана; предполагается, что гены одной группы сцепления всегда передаются вместе).

Случайным образом (50/50) выбирается, какую из двух родительских хромосом передать — «первую» или «вторую».

Для всех генов из списка из таблицы «Генотип» извлекается аллель, соответствующий выбранной хромосоме (если выбрана «первая» — берётся `'ID_аллеля_1_FK'`, если «вторая» — `'ID_аллеля_2_FK'`).

Идентификаторы аллелей конкатенируются в строку через запятую и возвращаются вызывающему коду

С точки зрения игрока: обеспечивает реализацию сцепленного наследования — группы признаков, которые в природе часто наследуются вместе. В интерфейсе может визуальнo отображаться, какие гены сцеплены, что помогает прогнозировать результаты скрещиваний.

20. Функция `calculate_punnett_probabilities(p_parent1_id, p_parent2_id, p_gene_id)` RETURN SYS_REFCURSOR

Общая суть работы: рассчитывает вероятности всех возможных генотипов потомка для одного гена по решётке Пеннета.

С точки зрения внутренней логики: определяет набор аллелей от каждого родителя (по два), строит все 4 комбинации, группирует одинаковые генотипы (порядок аллелей не важен), вычисляет вероятность как количество одинаковых комбинаций / 4. Возвращает курсор с колонками `allele1_id`, `allele2_id`, `probability`.

С точки зрения игрока: может использоваться в интерфейсе для предсказания вероятности появления нужного признака у потомка до скрещивания.

21. Процедура `crossbreed(p_lab_id, p_parent1_id, p_parent2_id, p_offspring_name, p_offspring_id)`

Общая суть работы: скрещивание двух родительских особей с получением потомка, чей генотип формируется по правилам менделевского наследования с учётом сцепленных генов.

С точки зрения внутренней логики:

Создание записи потомка

В таблицу «Существа» добавляется новая строка с именем `'p_offspring_name'` и текущей лабораторией. Полученный `'p_offspring_id'` используется далее.

Подготовка списка генов и групп сцепления

Из таблицы «Гены» выбираются все записи (ID гена, ID группы сцепления). Группировка происходит по полю `'linkage_group'`:

Гены, у которых `'linkage_group IS NOT NULL'`, считаются сцепленными и обрабатываются единым блоком.

Гены с `linkage_group IS NULL` обрабатываются независимо.

Для каждой группы сцепления (уникальное значение `'linkage_group'`):

Вызывается `get_linked_allele_set(p_parent1_id, linkage_group)`, возвращающая строку ID аллелей от первого родителя.

Вызывается `get_linked_allele_set(p_parent2_id, linkage_group)`, возвращающая строку ID аллелей от второго родителя.

Строки разбиваются на отдельные идентификаторы, которые в порядке следования генов в группе сопоставляются с соответствующими генами.

Для каждого гена группы в таблицу «Генотип» вставляется запись: `'ID_аллеля_1_FK' = аллель от родителя 1`, `'ID_аллеля_2_FK' = аллель от родителя 2`.

Таким образом, все гены одной группы сцепления наследуются как единое целое, без кроссинговера.

Обработка независимых генов: для каждого гена, не входящего ни в одну группу сцепления ('linkage_group IS NULL'):

allele1 := get_inherited_allele(p_parent1_id, gene_id)` — случайный аллель от первого родителя.

allele2 := get_inherited_allele(p_parent2_id, gene_id)` — случайный аллель от второго родителя.

- В таблицу «Генотип» вставляется запись с полученными аллелями.

В таблицу «Эксперименты» добавляется запись с типом 'CROSS', ссылками на родителей и полученного потомка. Вызывается процедура для автоматического выполнения заданий, если потомок удовлетворяет условиям. При необходимости агрегатные счётчики лаборатории актуализируются вызовом 'get_lab_stats' (или они будут пересчитаны при следующем запросе статистики).

С точки зрения игрока: при выборе двух родителей и запуске скрещивания игрок получает потомка, чьи признаки являются случайной, но закономерной комбинацией родительских. Сцепленные гены передаются блоками, независимые — по отдельности, что соответствует классической генетике. Результат может быть как предсказуемым (при анализе групп сцепления), так и неожиданным, стимулируя эксперименты.

22. Процедура rename_creature(p_creature_id, p_new_name)

Общая суть работы: изменение имени существа.

С точки зрения внутренней логики: UPDATE Существа SET Имя_существа = p_new_name WHERE ID_существа = p_creature_id;

С точки зрения игрока: позволяет давать особям осмысленные названия.

23. Процедура `apply_mutation(p_creature_id, p_mutation_id)`

Общая суть работы: применение купленной мутации к существу (изменение генотипа согласно правилам мутации).

С точки зрения внутренней логики: загружается описание мутации – какие гены изменяются и на какие аллели. Для указанных генов производится замена одного или обоих аллелей в таблице «Генотип». Записывается эксперимент с типом 'MUTATION'. Уменьшается запас мутации в таблице «Мутации_в_лаборатории».

С точки зрения игрока: после покупки мутации игрок может применить её к выбранному существу, что изменит один или несколько его признаков. Это может быть необходимо для выполнения сложных заданий или создания уникальных видов.

24. Функция `buy_mutation(p_lab_id, p_mutation_id)` RETURN BOOLEAN

Общая суть работы: покупка мутации в магазине.

С точки зрения внутренней логики: проверяется, достаточно ли средств в лаборатории (сравнение Кошелек со Стоимость_мутации). Если достаточно – средства списываются, и в таблице «Мутации_в_лаборатории» увеличивается количество данного типа мутаций (если записи не было – она создаётся). Возвращается TRUE. Иначе возвращается FALSE.

С точки зрения игрока: игрок может выбрать мутацию из магазина (список возвращается функцией `show_mutation_shop`), купить её за игровую валюту и затем применить к существу.

25. Процедура `make_experiment(p_lab_id, p_parent1_id, p_parent2_id, p_mutation_id, p_offspring_name, p_offspring_id)`

Общая суть работы: главный эксперимент – скрещивание с последующим применением мутации (если указана).

С точки зрения внутренней логики: сначала вызывается `crossbreed` для создания потомка. Если передан `p_mutation_id` (не NULL), то проверяется

наличие мутации в лаборатории; если её нет, то мутация не применяется. Затем к потомку применяется `apply_mutation`. Возвращается ID созданного существа.

С точки зрения игрока: позволяет за один шаг получить потомка и сразу применить к нему мутагенное воздействие. Удобно для ускоренного создания существ с нужными признаками.

26. Процедура `show_tasks(p_lab_id)`

Общая суть работы: вывод списка заданий лаборатории с их статусом (ACTIVE/COMPLETED).

С точки зрения внутренней логики: выполняется JOIN таблиц «Лабораторные_задания» и «Задания», результат выводится через DBMS_OUTPUT.

С точки зрения игрока: в текстовом интерфейсе можно увидеть, какие задания доступны, какие уже выполнены. В GUI используется курсор или отдельный запрос.

27. Функция `check_task(p_lab_id, p_task_id, p_creature_id) RETURN INTEGER`

Общая суть работы: проверить, удовлетворяет ли генотип переданного существа требованиям конкретного задания.

С точки зрения внутренней логики: из таблицы «Маркер задания» выбираются все идентификаторы аллелей, которые должны присутствовать в генотипе для выполнения задания. Затем в таблице «Генотип» для указанного существа проверяется, что каждый требуемый аллель встречается хотя бы в одном из двух аллельных слотов (ID_аллеля_1_FK или ID_аллеля_2_FK) для соответствующего гена. Если все необходимые аллели найдены, возвращается 1, иначе 0.

С точки зрения игрока: напрямую не вызывается. Результат используется системой для автоматического определения, выполнено ли задание.

28. Процедура `complete_task(p_lab_id, p_task_id, p_creature_id)`

Общая суть работы: зафиксировать выполнение задания, начислить награду и уведомить игрока.

С точки зрения внутренней логики: вызывается только в случае успешной проверки `check_task`. Выполняются следующие действия: из таблицы «Задания» считываются `Стоимость_задания` и `Рейтинг_задания`. В таблице «Лабораторные задания» строка с данным `p_task_id` и `p_lab_id` переводится в статус 'COMPLETED'. Увеличиваются кошелек и рейтинг лаборатории в таблице «Лаборатории» на соответствующие величины. Для немедленной обратной связи вызывается `DBMS_OUTPUT.PUT_LINE` с текстом уведомления (в текстовом интерфейсе) или, при наличии GUI-обёртки, через процедуру отправки уведомлений. При необходимости обновляются агрегатные счётчики лаборатории вызовом `get_lab_stats`.

С точки зрения игрока: никаких действий не требуется. Как только в коллекции появляется существо с нужными признаками, игрок получает оповещение о выполненном задании, видит увеличение монет и рейтинга, а в списке заданий оно помечается как выполненное.

29. Процедура `apply_mutagen(p_creature_id, p_mutagen_type, p_new_creature_id)`

Общая суть работы: воздействие мутагеном (радиация/химикаты) на конкретное существо, изменение случайного или заданного гена.

С точки зрения внутренней логики: если `p_target_gene` не указан, выбирается случайный ген из генотипа существа. В зависимости от типа мутагена определяется допустимый новый аллель (для радиации – полностью случайный, для химикатов – близкий по типу). Один из аллелей заменяется на новый. Добавляется запись в «Эксперименты» с типом 'MUTAGEN'. Увеличивается счётчик экспериментов лаборатории.

С точки зрения игрока: во вкладке «Мутагенные эксперименты» игрок выбирает существо, мутаген и проводит эксперимент.

30. Процедура show_mutation_history(p_lab_id)

Общая суть работы: вывод истории экспериментов, связанных с мутациями и мутагенами.

С точки зрения внутренней логики: выбираются записи из «Эксперименты» с типом 'MUTATION' или 'MUTAGEN', выполняется JOIN с «Существа» и «Мутации», результат выводится в консоль.

С точки зрения игрока: можно просмотреть, какие мутации и когда применялись, какие гены изменялись. Полезно для анализа удачных/неудачных экспериментов.

31. Функция show_mutation_shop RETURN SYS_REFCURSOR

Общая суть работы: возвращает курсор со всеми доступными для покупки мутациями из таблицы «Мутации».

С точки зрения внутренней логики: открывается курсор SELECT ID_мутации, Название_мутации, Стоимость_мутации, Рейтинг_мутации FROM Мутации ORDER BY Стоимость_мутации.

С точки зрения игрока: в GUI отображается магазин, где можно выбрать мутацию, посмотреть её цену и рейтинг, затем купить.

32. Процедура generate_starting_creatures(p_lab_id)

Общая суть работы: генерация 30 стартовых существ: для каждого из 6 базовых типов создаётся 5 особей.

С точки зрения внутренней логики: внешний цикл для p_species_type от 1 до 6, внутренний цикл для p_variant от 1 до 5. Для каждой комбинации вызывается create_creature_of_type. После завершения обновляется поле Кол_сущетв в «Лаборатории».

С точки зрения игрока: при создании новой лаборатории игрок сразу получает 6 видов (хрящевые рыбы, костные рыбы, ракообразные, моллюски, черепахи, млекопитающие) по 5 особей каждого. Этого достаточно для начала экспериментов.

33. Процедура `create_creature_of_type(p_lab_id, p_species_type, p_variant, p_creature_id)`

Общая суть работы: создаёт одно существо заданного биологического типа с уникальным генотипом. Гены для него отбираются по столбцу `тип_существа`: берутся все универсальные гены (0) и гены, у которых `тип_существа = p_species_type`. Для каждого отобранного гена случайным образом выбираются два аллеля.

С точки зрения внутренней логики: формируется имя существа в зависимости от `p_species_type` и `p_variant`, например 'Хрящевая рыба #2'.

В таблицу «Существа» вставляется запись с именем, `ID_лаборат_FK = p_lab_id`, столбцы фенотипа пока пусты.

Из таблицы «Гены» выбираются все записи, где `тип_существа = 0` или `тип_существа = p_species_type`.

Для каждого отобранного гена: из таблицы «Аллель» извлекаются все аллели, относящиеся к этому гену (`ID_гена_FK = gene_id`).

Случайным образом (равновероятно) выбираются два аллеля (допустимы одинаковые).

В таблицу «Генотип» вставляется запись: (`p_creature_id`, `gene_id`, `allele1_id`, `allele2_id`).

Вызывается `get_phenotype(p_creature_id)`, которая заполняет столбцы фенотипа в таблице «Существа» на основе сформированного генотипа.

С точки зрения игрока: каждое стартовое существо наследует признаки, разрешённые его биологическому типу. Универсальные гены (цвет, размер и т.п.) присутствуют у всех, что создаёт разнообразие внутри типов.

34. Процедура `delete_lab(p_lab_id INTEGER)`

Общая суть: удаление лаборатории и всех связанных с ней данных.

Внутренняя логика: удаляются записи из таблиц «Генотип», «Существа», «Эксперименты», «Лабораторные_задания»,

«Мутации_в_лаборатории», затем сама лаборатория. Действие выполняется только для лабораторий, принадлежащих текущему пользователю.

Для игрока: возможность полностью удалить ненужную лабораторию из списка.